

NeuroMF: Latent MeanFlow for One-Step 3D Brain MRI Synthesis

Technical Report — Detailed Scientific Summary

Mario Pascual-González

February 2026

Phases 0–5 complete (code); Phase 4 best model trained and evaluated; Phase 5 generation pipeline ready

Contents

1	Motivation and Problem Statement	4
1.1	Clinical Need	4
1.2	Gap in the Literature	4
1.3	Approach Overview	4
2	Theoretical Foundations	4
2.1	Flow Matching Preliminaries	4
2.2	MeanFlow: Average Velocity for 1-Step Generation	5
2.3	The MeanFlow Identity and Self-Consistency	5
2.4	Improved MeanFlow (iMF) and the Dual-Head Architecture	5
2.5	x-Prediction Reparameterisation	6
2.6	Latent Space Formulation	6
3	Architecture Design	6
3.1	Frozen MAISI VAE (Encoder-Decoder)	6
3.2	MeanFlow UNet (Generative Model)	7
3.3	Dual-Time Conditioning	7
3.4	v-Head (Auxiliary Tangent Predictor)	8
4	Loss Function and Training Objective	9
4.1	Combined iMF Dual-Head Loss	9
4.2	Adaptive Weighting	9
4.3	Per-Channel L_p Loss	10
4.4	JVP Strategies	10
5	Data Pipeline	10
5.1	Dataset	10
5.2	Preprocessing Pipeline	11
5.3	Latent Pre-Computation (Phase 1)	11

5.4	Latent Augmentation	11
6	Training Protocol	12
6.1	Algorithm (One Training Step)	12
6.2	Time Sampling	12
6.3	Optimiser and Schedule	12
6.4	Compute Budget	12
6.5	EMA (Exponential Moving Average)	12
6.6	Divergence Monitoring	13
7	Sampling and Generation	13
7.1	One-Step Sampling (1-NFE)	13
7.2	Multi-Step Euler Sampling	13
7.3	Full Generation Pipeline (Phase 5)	14
8	Experimental Results	14
8.1	Toy Validation (Phase 2): MeanFlow on Flat Torus	14
8.2	Main Training (Phase 4): Best Model	14
8.3	FID Progression	15
8.4	Training Dynamics	15
8.5	Generated Sample Statistics	15
8.6	NFE Quality Analysis	16
8.7	Spectral Analysis	16
8.8	SWD (Sliced Wasserstein Distance)	16
8.9	Key Ablation Insights	16
9	Evaluation Protocol	17
9.1	Image Quality Metrics	17
9.2	Spectral Analysis	17
9.3	Morphological Assessment	17
9.4	NFE Consistency Analysis	17
10	Implementation Details	17
10.1	Software Stack	17
10.2	Gated Phase System	18
10.3	Compute Infrastructure	18
11	Novel Contributions	18
12	Current Status and Next Steps	19
12.1	Current State (February 2026)	19
12.2	Key Findings	19

12.3 Open Questions	19
-------------------------------	----

1 Motivation and Problem Statement

1.1 Clinical Need

Generative models for 3D brain MRI synthesis serve three clinical purposes: (i) data augmentation for rare pathologies with scarce training data, (ii) synthetic cohorts for privacy-preserving research, and (iii) counterfactual generation for explainability. The critical bottleneck in existing methods is **sampling cost**: state-of-the-art approaches (DDPM, flow matching, rectified flow) require 5–1000 network evaluations per volume, making large-scale synthesis prohibitively slow.

1.2 Gap in the Literature

No prior work has applied MeanFlow—or any 1-step flow-based model—to 3D medical image synthesis. The closest works are summarised in Table 1.

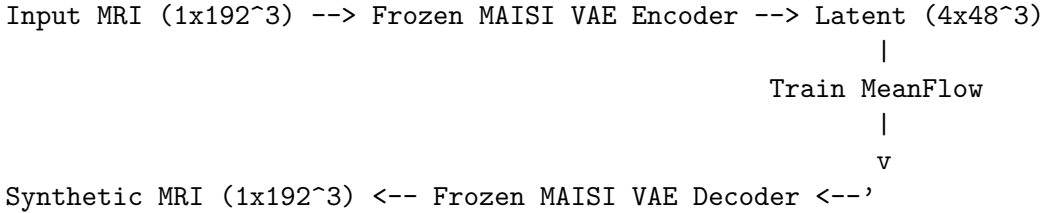
Table 1: Comparison with related generative methods for brain MRI.

Method	Space	NFE	Paradigm	Domain
MAISI-v2 [5]	Latent	5–50	Rectified Flow	3D CT/MRI
MOTFM [7]	Pixel	10–50	OT Flow Matching	3D Brain MRI
Med-DDPM [8]	Pixel	1000	DDPM	3D Brain MRI
pMF [3]	Pixel	1	Progressive MeanFlow	2D Natural Images
NeuroMF (ours)	Latent	1	MeanFlow (iMF dual-head)	3D Brain MRI

Our work fills the intersection: **1-step** + **latent** + **3D** + **medical**.

1.3 Approach Overview

NeuroMF trains a MeanFlow model in the latent space of a frozen MAISI 3D VAE:



At inference, a **single forward pass** through the MeanFlow network generates a complete 3D brain MRI volume. The network learns the *average velocity* of a probability flow ODE, which by construction encodes the entire transport from noise to data in one evaluation.

2 Theoretical Foundations

2.1 Flow Matching Preliminaries

Flow matching [9, 10] defines a probability path between data and noise via linear interpolation:

$$z_t = (1 - t) z_0 + t \varepsilon, \quad z_0 \sim p_{\text{data}}, \quad \varepsilon \sim \mathcal{N}(0, I), \quad t \in [0, 1] \quad (1)$$

where $t=0$ is data and $t=1$ is noise. The conditional velocity field is $v_c(z_t, t) = \varepsilon - z_0$, and a neural network is trained to match this field:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{z_0, \varepsilon, t} [\|v_\theta(z_t, t) - v_c\|^2] \quad (2)$$

Limitation. Sampling requires integrating the learned velocity field from $t=1$ to $t=0$ via numerical ODE solvers, requiring $K \geq 5$ network evaluations even with trajectory straightening (rectified flow).

2.2 MeanFlow: Average Velocity for 1-Step Generation

MeanFlow [1] replaces the instantaneous velocity $v(z_t, t)$ with the **average velocity** over an interval $[r, t]$:

$$u(z_t, r, t) = \frac{1}{t-r} \int_r^t v(z_s, s) ds \quad (3)$$

If the average velocity from $t=0$ to $t=1$ is known exactly, the entire flow can be computed in one step:

$$z_0 = z_1 - u_\theta(z_1, 0, 1) \quad [1\text{-NFE generation}] \quad (4)$$

2.3 The MeanFlow Identity and Self-Consistency

The average velocity u must satisfy a self-consistency condition. Differentiating the integral definition and applying the chain rule yields the **MeanFlow Identity**:

$$v(z_t, t) = u(z_t, r, t) + (t-r) \left[\frac{\partial u}{\partial z_t} v(z_t, t) + \frac{\partial u}{\partial t} \right] \quad (5)$$

The right-hand side, evaluated with the neural approximation, gives the **compound velocity**:

$$V_\theta = u_\theta + (t-r) \cdot \text{sg}[\text{JVP}(u_\theta; (z_t, t, r); (\tilde{v}, 1, 0))] \quad (6)$$

where $\text{sg}[\cdot]$ denotes stop-gradient and the JVP (Jacobian-Vector Product) is:

$$\text{JVP} = \frac{\partial u}{\partial z_t} \tilde{v} + \frac{\partial u}{\partial t} \cdot 1 + \frac{\partial u}{\partial r} \cdot 0 \quad (7)$$

This JVP is computed in $O(d)$ time via forward-mode automatic differentiation (`torch.func.jvp`), avoiding the $O(d^2)$ cost of constructing the full Jacobian. The tangent vector $\tilde{v}$ is the model’s own estimate of the instantaneous velocity at $r=t$: $\tilde{v} = u_\theta(z_t, t, t)$.

2.4 Improved MeanFlow (iMF) and the Dual-Head Architecture

The original MeanFlow uses the ground-truth velocity $v_c = \varepsilon - z_0$ as the JVP tangent. **Problem:** this creates a dependency on data at inference time (where z_0 is unavailable). The improved MeanFlow (iMF) [2] resolves this by using the model’s own prediction as the tangent, making V a function of z_t alone.

The dual-head extension (our architecture, inspired by iMF) introduces two output heads on a shared UNet backbone:

- **u-head** (main): Predicts the average velocity u (or equivalently, $\hat{x}$ in x-prediction mode). Used at inference.
- **v-head** (auxiliary): Predicts the instantaneous velocity, directly supervised against v_c . Provides the JVP tangent during training. **Disabled at inference** (zero cost).

The v-head solves a critical practical problem: without it, the tangent comes from the u-head’s own prediction, which is poor in early training, creating a bootstrapping problem that causes loss divergence. The v-head receives direct supervision ($\|v - v_c\|^p$), providing high-quality tangents from the first epoch. In our training, the v-head cosine alignment $\cos(\hat{v}, v_c)$ reached 0.39 by epoch 50—85% of its final value of 0.44.

2.5 x-Prediction Reparameterisation

Instead of directly outputting the average velocity u , the network outputs a denoised data estimate $\hat{x}$ (x-prediction, following pMF [3]):

$$\hat{x} = f_\theta(z_t, r, t), \quad u_\theta = \frac{z_t - \hat{x}}{\max(t, t_{\min})} \quad (8)$$

Justification (manifold hypothesis). The x-prediction target lies on the data manifold, which has low intrinsic dimensionality. The velocity u spans a higher-dimensional space. For architectures with a bottleneck (UNet encoder-decoder), predicting a low-dimensional target ($\hat{x}$) is easier than predicting a high-dimensional one (u).

Quantitative criterion (pMF Table 2): x-prediction dominates when $d_{\text{input}}/d_{\text{bottleneck}} > 1$. For our 3D UNet:

$$\frac{d_{\text{input}}}{d_{\text{bottleneck}}} = \frac{4 \times 48^3}{512 \times 6^3} = \frac{442,368}{110,592} \approx 4 \quad (9)$$

This is firmly in the x-prediction regime.

At inference (1-NFE). With x-prediction, 1-step sampling simplifies to:

$$z_0 = f_\theta(\varepsilon, r=0, t=1) \quad (10)$$

No velocity-to-data conversion is needed—the model directly outputs the synthetic latent.

2.6 Latent Space Formulation

All of the above operates identically in latent space. The computational advantages are:

1. **JVP cost reduction:** Latent dimension $d = 4 \times 48^3 = 442,368$ vs pixel space $d = 192^3 = 7,077,888$ —a **16× reduction** in JVP compute per iteration.
2. **Memory reduction:** The UNet operates on 48^3 feature maps vs 192^3 —approximately **64× reduction** in activation memory per JVP pass.
3. **Training efficiency:** The latent space is pre-computed (Phase 1), so VAE encode/decode cost is amortised across all training epochs.

3 Architecture Design

3.1 Frozen MAISI VAE (Encoder-Decoder)

The MAISI VAE [4] is a 3D variational autoencoder with adversarial training, pre-trained on ~55K CT+MRI volumes. We use it as a **frozen foundation model**—all parameters are locked, and it is never fine-tuned. Key properties are listed in Table 2.

Reconstruction quality (Phase 0, 20 IXI volumes): Mean SSIM = 0.9213, Mean PSNR = 30.86 dB.

Scale factor. The decode operation divides by `scale_factor` before passing to the decoder: $\hat{x} = \text{decoder}(z / 0.9624)$. The `scale_factor` was extracted from the MAISI diffusion checkpoint (`diff_unet_3d_rflow-mr.pt["scale_factor"]`), not the VAE checkpoint.

Table 2: MAISI VAE properties.

Property	Value
Parameters	20,944,897 (~ 21 M)
Encoder stages	3 levels of $2\times$ strided 3D conv
Latent channels	4 (with KL regularisation)
Spatial compression	$4\times$ per axis: $(1, 192^3) \rightarrow (4, 48^3)$
Attention	None (all <code>attention_levels=false</code>)
Training losses	L_1 + LPIPS + PatchGAN + KL
Checkpoint format	Wrapped in "unet_state_dict" key
Scale factor	0.96240234375
Memory optimisation	<code>num_splits</code> for chunk-based processing

Latent distribution quality. Encoding all 6,471 volumes (train + val + test) confirms the latent space is well-regularised (Table 3).

Table 3: Per-channel latent statistics ($n = 6,471$ encoded volumes).

Channel	Mean	Std	Skewness	Kurtosis
0	−0.053	0.970	+0.105	−0.123
1	−0.185	1.019	+0.002	−0.019
2	−0.051	0.970	+0.045	+0.099
3	+0.001	1.011	+0.069	+0.144

The distributions are near-Gaussian: skewness $|\gamma| < 0.11$, excess kurtosis $|\kappa| < 0.15$, and standard deviations cluster around 0.97–1.02. Cross-channel correlations are negligible (max off-diagonal $|r| = 0.046$), confirming the KL regularisation produces approximately independent, unit-variance channels.

3.2 MeanFlow UNet (Generative Model)

The MeanFlow UNet (`MAISIUNetWrapper`) uses the **same architecture** as the MAISI diffusion UNet but with random initialisation and custom dual-time conditioning. Key properties are listed in Table 4.

Flash attention incompatibility. `torch.func.jvp` uses forward-mode AD, which requires the computation graph to be fully traceable. Flash attention’s fused CUDA kernels are opaque to PyTorch’s AD system. Disabling flash attention adds $\sim 10\%$ latency but enables exact JVP.

Gradient checkpointing incompatibility. Gradient checkpointing re-executes forward passes during backward, which conflicts with the forward-mode AD tape maintained by `torch.func.jvp`.

3.3 Dual-Time Conditioning

MeanFlow requires conditioning on two time variables: t (current time) and r (interval lower bound). We implement three conditioning modes and select `t_h` based on ablation (Table 5).

The `t_h` mode conditions on both the absolute time t and the interval width $h = t - r$. The original MeanFlow paper (Table 1c) reports FID 61.06 for (t, h) vs 63.13 for h -only on ImageNet.

Implementation. Continuous time values in $[0, 1]$ are scaled by 1000 before computing sinusoidal embeddings. After sinusoidal encoding, each embedding passes through a 2-layer MLP

Table 4: MeanFlow UNet properties.

Property	Value
Backbone	MONAI DiffusionModelUNet (3D)
Total parameters	~ 178 M
Channels per level	[64, 128, 256, 512]
Attention levels	[F, F, T, T]—only at 12^3 and 6^3
Attention heads	32 channels/head at attention levels
ResBlocks per level	2
GroupNorm groups	32
Transformer layers	1 per attention level
Flash attention	Disabled (<code>torch.func.jvp</code> compat.)
Gradient checkpointing	Disabled (exact JVP compat.)
ResBlock downsampling	Strided convolution
Prediction type	x-prediction ($\hat{x}$)

Table 5: Conditioning modes. Selected mode in **bold**.

Mode	Inputs	Embedding	Source
dual	(r, t)	$\sin(r) + \sin(t)$ via separate MLPs	pMF
h	$h = t - r$	$\sin(h)$ via UNet MLP	iMF
t_h	$(t, h=t-r)$	$\sin(t) + \sin(h)$ via two MLPs	MF Tab. 1c

(Linear(64, 256) $\rightarrow$ SiLU $\rightarrow$ Linear(256, 256)), and the two are summed:

$$\text{emb} = \text{MLP}_t(\sin(t \cdot 1000)) + \text{MLP}_h(\sin(h \cdot 1000)) \quad (11)$$

3.4 v-Head (Auxiliary Tangent Predictor)

The v-head is a lightweight auxiliary output path branching from the UNet’s final feature map (before the main output convolution):

```

Shared Backbone (B, 64, 48, 48, 48)
|-- u-head: UNet out conv --> (B, 4, 48, 48, 48) [main]
|-- v-head: ResBlock -> GN -> SiLU -> Conv3d --> (B, 4, 48, 48, 48)

```

Table 6: v-Head properties.

Property	Value
ResBlocks	1
Parameters	~ 228 K ($< 0.13\%$ of total 178 M)
Initialisation	Final Conv3d zero-initialised
Inference cost	Zero (output discarded at sampling)

Zero initialisation rationale. The v-head starts at identically zero, so u-head training is unaffected at initialisation. Our training confirms: $\cos(\hat{v}, v_c)$ rises from 0.17 (epoch 0) to 0.39 (epoch 50) to 0.44 (epoch 689).

4 Loss Function and Training Objective

4.1 Combined iMF Dual-Head Loss

The training loss has two independently-weighted components:

$$\mathcal{L} = \mathcal{L}_u^{\text{weighted}} + \mathcal{L}_v^{\text{weighted}} \quad (12)$$

where:

- $\mathcal{L}_u$ (**compound velocity loss**): $\|V_\theta - v_c\|_p^p$ — enforces MeanFlow self-consistency.
- $\mathcal{L}_v$ (**tangent supervision loss**): $\|\hat{v} - v_c\|_p^p$ — directly supervises the v-head.

Both losses target the same conditional velocity $v_c = \varepsilon - z_0$.

4.2 Adaptive Weighting

Each loss component is independently normalised by its own magnitude:

$$w_u = (\mathcal{L}_u^{\text{raw}}|_{\text{detach}} + \varepsilon_{\text{norm}})^{p_{\text{norm}}}, \quad \mathcal{L}_u^{\text{weighted}} = \frac{\mathcal{L}_u^{\text{raw}}}{w_u} \quad (13)$$

$$w_v = (\mathcal{L}_v^{\text{raw}}|_{\text{detach}} + \varepsilon_{\text{norm}})^{p_{\text{norm}}}, \quad \mathcal{L}_v^{\text{weighted}} = \frac{\mathcal{L}_v^{\text{raw}}}{w_v} \quad (14)$$

With $\varepsilon_{\text{norm}}=1.0$ and $p_{\text{norm}}=1.0$:

$$w = \mathcal{L}^{\text{raw}} + 1.0, \quad \mathcal{L}^{\text{weighted}} = \frac{\mathcal{L}^{\text{raw}}}{\mathcal{L}^{\text{raw}} + 1.0} \quad (15)$$

This is a form of **logarithmic loss normalisation**: the effective loss is bounded in $[0, 1]$ and varies slowly with raw loss magnitude. At convergence (epoch 388, best FID), the loss decomposition shows this balancing in action (Table 7).

Table 7: Loss decomposition at best FID (epoch 388).

Component	Raw Loss	Share
FM loss ($r = t$, flow matching)	715,432	10.3%
MF loss ($r < t$, self-consistency)	6,254,997	89.7%
v-head loss	714,646	—
u-head loss (compound V)	3,485,215	—

The MF loss is $\sim 9\times$ larger than the FM loss, which is expected: the compound velocity V must also capture the JVP correction term, a harder target than the direct velocity at $r=t$. The adaptive weighting equalises their gradient contributions to ~ 1.0 each.

Why $\varepsilon_{\text{norm}}=1.0$? Through Phase 4c debugging, we discovered that $\varepsilon_{\text{norm}}=0.01$ caused catastrophic gradient amplification for samples with small raw loss. The weight $1/(\mathcal{L} + 0.01)$ can reach $100\times$, creating a positive feedback loop.

Why $p_{\text{norm}}=1.0$? With $p_{\text{norm}}=0.5$, the weight becomes $w = \sqrt{\mathcal{L} + 1.0}$, which under-normalises large losses, leading to a $1000\times$ gradient explosion (Phase 4e).

4.3 Per-Channel L_p Loss

The base loss function computes a per-channel, spatially-summed L_p norm:

$$\ell_p(\hat{y}, y) = \sum_{c=1}^C \sum_{\mathbf{x}} |\hat{y}_{c,\mathbf{x}} - y_{c,\mathbf{x}}|^p \quad (16)$$

For the best model: $p=2.0$ (standard L_2 loss). The per-channel L_p framework supports ablation over $p \in \{1.0, 1.25, 1.5, 1.75, 2.0, 2.5, 3.0\}$ and per-channel weight vectors, extending the SLIM-Diff per-channel loss [6] from pixel-space DDPM to latent-space MeanFlow.

Key scientific question. Does the optimal L_p exponent from pixel space ($p=1.5$ for images, $p=2.0$ for masks in SLIM-Diff) transfer through the VAE nonlinearity to latent space?

4.4 JVP Strategies

Two strategies are implemented:

Exact JVP (`torch.func.jvp`):

$$u, \frac{du}{dt} = \text{jvp}(u_\theta; (z_t, t, r); (\tilde{v}, 1, 0)), \quad V = u + (t - r) \cdot \text{sg}\left[\frac{du}{dt}\right] \quad (17)$$

- Most accurate, $O(d)$ cost.
- Requires: no in-place ops, no flash attention, no gradient checkpointing.
- Memory: ~ 20 GB activation per sample at batch = 2 on A100-40GB.

Finite Difference JVP (FD-JVP):

$$u_h = u_\theta(z_t + h\tilde{v}, t + h, r), \quad \frac{du}{dt} \approx \frac{u_h^{(\text{fp32})} - u^{(\text{fp32})}}{h}, \quad V = u + (t - r) \cdot \text{sg}\left[\frac{du}{dt}\right] \quad (18)$$

- Step size $h = 10^{-3}$.
- FP32 subtraction to avoid bf16 catastrophic cancellation.
- Lower memory (perturbed pass is `no_grad`).

Critical incompatibility (Phase 4f). x-prediction + FD-JVP is **numerically unstable**. The x-to-u conversion $u = (z_t - \hat{x})/t$ has a $1/t$ singularity. When FD-JVP computes $(u(t+h) - u(t))/h$, the result involves $O(1/t)$ terms divided by $h=0.001$:

$$\frac{du}{dt} \sim O\left(\frac{1}{t^2 \cdot h}\right) \rightarrow \infty \quad \text{as } t \rightarrow t_{\min} \quad (19)$$

With exact JVP, the $1/t$ factor is analytically differentiated, yielding stable gradients.

Rule: x-pred + exact JVP = stable u-pred + FD-JVP = stable x-pred + FD-JVP = explosion

For dual-head models, exact JVP uses `has_aux=True` to capture the v-head output alongside the JVP computation, avoiding redundant forward passes:

$$(u, du/dt, \hat{v}) = \text{jvp}(u_{\text{with_v_aux}}, (z_t, t, r), (\tilde{v}, 1, 0), \text{has_aux}=\text{True}) \quad (20)$$

5 Data Pipeline

5.1 Dataset

We use 8 datasets from FOMO-60K, a large-scale preprocessed brain MRI collection (Table 8).

Table 8: Dataset composition from 8 FOMO-60K subsets.

Dataset	Train Subj.	Train Scans	Val Scans	Test Scans
PT001_OASIS1	352	1,414	170	96
PT002_OASIS2	71	682	82	37
PT005_IXI	494	494	58	29
PT007_NIMH	212	428	48	23
PT008_DLBS	394	807	109	51
PT011_MBSR	125	295	36	16
PT012_UCLA	106	106	13	6
PT015_NKI	722	1,245	158	68
Total	2,476	5,471	674	326

Many subjects have multiple sessions (longitudinal scans), yielding 5,471 training scans from 2,476 subjects—approximately $4\times$ more data than the initially planned $\sim 1,172$ scans from 3 datasets. All volumes are T1-weighted, skull-stripped, RAS-oriented, and co-registered. The split is stratified by dataset with a fixed seed of 42 for reproducibility.

5.2 Preprocessing Pipeline

1. `LoadImaged` (MONAI)
2. `EnsureChannelFirstd`
3. `Spacingd`(pixdim=1.0 mm isotropic, bilinear)
4. `ScaleIntensityRangePercentilesd`(0%–99.5% $\rightarrow$ $[0, 1]$)
5. `CropForegroundd`(margin=4) — brain-centred crop
6. `ResizeWithPadOrCropd`(192, 192, 192) — pad to target
7. `EnsureTyped`(dtype=float32)

Resolution choice (192^3 at 1.0 mm isotropic). Selected after quantitative analysis of brain extent across 30 FOMO-60K subjects. Brain AP extent reaches 193 mm; 128^3 and 160^3 both clip frontal/occipital cortex. `CropForegroundd` centres the crop on brain tissue, achieving 100% brain coverage for all subjects.

Latent shape: $192/4 = 48$ per spatial axis, giving latents of shape (4, 48, 48, 48).

5.3 Latent Pre-Computation (Phase 1)

All training volumes are pre-encoded through the frozen MAISI VAE and stored in HDF5 shards. A total of 6,471 volumes were encoded (~ 11.5 GB storage).

Latent statistics are computed online during encoding via `LatentStatsAccumulator` (sum-of-powers method in float64): per-channel mean, std, skewness, kurtosis, min, max, and a 4×4 cross-channel Pearson correlation matrix.

5.4 Latent Augmentation

Three augmentation transforms operate directly on latents during training (Table 9).

Approximately 68% of training samples are augmented per epoch ($\sim 3,720$ out of 5,471).

Table 9: Latent augmentation transforms.

Transform	Prob.	Params	Rationale
Flip depth axis	0.5	L-R in RAS	Bilateral symmetry
Gaussian noise	0.2	5% of per-ch. std	Encoding noise
Intensity scale	0.2	$\pm 5\%$	Scanner variation

6 Training Protocol

6.1 Algorithm (One Training Step)

Given a batch of pre-computed latents $\{z_0\}$ and the model with u-head and v-head:

1. Sample $\varepsilon \sim \mathcal{N}(0, I)$, same shape as z_0 .
2. Sample $t \sim \text{LogitNormal}(\mu=-0.4, \sigma=1.0)$, clamp to $[0.001, 1.0]$.
3. With probability 0.5: set $r = t$ (FM sample); otherwise: sample $r < t$ (MF sample).
4. Interpolate: $z_t = (1 - t)z_0 + t\varepsilon$.
5. Target: $v_c = \varepsilon - z_0$.
6. Tangent: $\tilde{v} = \text{v-head}(z_t, t, t)$ [no_grad].
7. JVP: $(u, du/dt, \hat{v}) = \text{jvp}(\text{dual_fn}, (z_t, t, r), (\tilde{v}, 1, 0))$ [exact, with aux].
8. Compound velocity: $V = u + (t - r) \cdot \text{sg}[du/dt]$.
9. Losses: $\mathcal{L}_u^{\text{raw}} = \|V - v_c\|_2^2$, $\mathcal{L}_v^{\text{raw}} = \|\hat{v} - v_c\|_2^2$.
10. Adaptive weighting: $\mathcal{L} = \mathcal{L}_u^{\text{raw}} / (|\mathcal{L}_u^{\text{raw}}| + 1) + \mathcal{L}_v^{\text{raw}} / (|\mathcal{L}_v^{\text{raw}}| + 1)$.
11. Backward, clip gradients ($\|\nabla\|_{\text{max}} = 1.0$), optimiser step.
12. EMA update ($\beta = 0.9999$).

6.2 Time Sampling

Times are drawn from a logit-normal distribution:

$$t = \sigma(\mathcal{N}(\mu=-0.4, \sigma=1.0)), \quad t \in [0.001, 1.0] \quad (21)$$

where $\sigma(\cdot)$ is the sigmoid function. This distribution is heavy near $t=0$ (near data), focusing training on the denoising regime.

`data_proportion = 0.5`. Half the batch has $r=t$ (FM samples where the JVP term vanishes) and half has $r<t$ (MF samples where self-consistency is enforced). Pure MF samples diverge; too many FM samples slow 1-NFE convergence.

6.3 Optimiser and Schedule

Hyperparameters are listed in Table 10.

6.4 Compute Budget

6.5 EMA (Exponential Moving Average)

An EMA model with decay $\beta = 0.9999$ is maintained throughout training:

$$\theta_{\text{EMA}} \leftarrow \beta \theta_{\text{EMA}} + (1 - \beta) \theta \quad (22)$$

All FID evaluations during training use EMA weights. The best model checkpoint is selected by FID computed from EMA-weight samples.

Table 10: Optimiser hyperparameters.

Hyperparameter	Value	Justification
Optimiser	AdamW	Standard for diffusion/flow models
Learning rate	10^{-4}	MeanFlow reference default
Weight decay	0.0	No regularisation with adaptive loss
Betas	(0.9, 0.95)	$\beta_2=0.95$: faster momentum adaptation
LR schedule	Cosine decay	$10^{-4} \rightarrow 5.6 \times 10^{-5}$ at early stop
Warmup	0 steps	Warmup destabilised MF loss (Phase 4c)
Gradient clip	$\ \nabla\ _{\max} = 1.0$	Prevents spike propagation
Mixed precision	bf16	Standard for A100

Table 11: Compute budget and actual training cost.

Property	Value
Hardware	$6 \times$ A100-SXM4-40GB (one DGX node)
Strategy	DDP (DistributedDataParallel)
Per-GPU batch	2 (JVP activation: ~ 20 GB)
Gradient accumulation	11 steps
Effective batch	$2 \times 6 \times 11 = 132$
Optimiser steps/epoch	41
Planned optimiser steps	61,500 ($1,500$ epochs $\times$ 41)
Actual optimiser steps	$\sim 28,980$ (early-stopped at epoch 690)
Mean epoch time	268.1 s (~ 4.5 min)
Total wall time	~ 51.4 hours (~ 2.1 days)

6.6 Divergence Monitoring

An EMA-smoothed raw loss monitor (decay 0.99, half-life ~ 69 steps) tracks training stability. Warnings fire at $3\times$ and $5\times$ the minimum EMA loss, with a grace period of 1000 steps. Early stopping is FID-based via `EvaluationCallback` with `patience = 10`.

7 Sampling and Generation

7.1 One-Step Sampling (1-NFE)

With x-prediction, 1-NFE sampling is a single forward pass:

$$\varepsilon \sim \mathcal{N}(0, I), \quad z_0 = f_\theta(\varepsilon, r=0, t=1) \quad (23)$$

The v-head output is discarded—only the u-head (main output) is used.

7.2 Multi-Step Euler Sampling

For comparison, multi-step Euler sampling divides $[1, 0]$ into K uniform steps:

$$t_i = 1 - \frac{i}{K}, \quad t_{i+1} = 1 - \frac{i+1}{K}, \quad \Delta t = t_i - t_{i+1} \quad (24)$$

$$\hat{x} = f_\theta(z, r=t_{i+1}, t=t_i), \quad u = \frac{z - \hat{x}}{\max(t_i, 0.05)}, \quad z \leftarrow z - \Delta t \cdot u \quad (25)$$

NFE levels tested: $\{1, 2, 5, 10\}$.

7.3 Full Generation Pipeline (Phase 5)

1. Pre-generate shared noise (for NFE-consistency analysis).
2. For each NFE level: generate latents, store in HDF5 (latents, seeds, timing).
3. Denormalise: $z_0 = \hat{z} \cdot \sigma_{\text{latent}} + \mu_{\text{latent}}$.
4. Decode: $\hat{x} = \text{VAE.decode}(z_0)$ [internally divides by `scale_factor`].
5. Clamp to $[0, 1]$; store decoded volumes in HDF5.

Shared noise protocol. The same noise tensor ε is used across all NFE levels for a given sample index, enabling direct comparison of how additional steps refine the output.

8 Experimental Results

8.1 Toy Validation (Phase 2): MeanFlow on Flat Torus

Before scaling to brain MRI, MeanFlow was validated on a flat torus in $\mathbb{R}^4$ (Table 12).

Table 12: Phase 2 toy validation results.

Metric	Result	Target
1-NFE torus distance	0.0892	<0.1
5-step torus distance	0.0271	<0.05
Angular KS p -value (θ_1)	0.374	>0.01
Angular KS p -value (θ_2)	0.575	>0.01
x-pred vs u-pred loss ratio	<1.2	<1.5

This confirmed: (i) the MeanFlow implementation correctly learns 1-step generation, (ii) the angular distribution is statistically indistinguishable from truth (KS test), and (iii) both x-prediction and u-prediction converge.

8.2 Main Training (Phase 4): Best Model

The best model uses x-prediction + exact JVP + (t, h) conditioning + v-head (1 ResBlock) + L_2 loss (Table 13).

Table 13: Best model configuration.

Property	Value
Prediction type	x-prediction
JVP strategy	Exact (<code>torch.func.jvp</code>)
Conditioning	t_h (condition on t and $h = t - r$)
v-head	1 ResBlock (~ 228 K params, <0.13%)
Loss norm	L_2 ($p=2$) with independent adaptive weighting
Best FID epoch	388 (step 16,338)
Early-stopped at	Epoch 690 (patience=10, 46% of planned 1,500)

8.3 FID Progression

FID is computed as a 2.5D metric using RadImageNet ResNet-50 features on central axial, coronal, and sagittal slices (Table 14).

Table 14: 2.5D FID progression during training (selected checkpoints).

Epoch	Step	FID _{avg}	FID _{xy}	FID _{yz}	Notes
9	420	46.76	45.12	46.68	First evaluation
28	1,218	28.40	17.25	43.63	Rapid initial drop
88	3,738	14.61	14.08	16.25	Approaching plateau
178	7,518	14.06	13.79	15.46	
268	11,298	12.94	12.77	14.06	
388	16,338	11.67	11.85	12.09	Best FID
628	26,418	11.92	12.03	12.33	
688	28,938	11.88	12.14	12.08	Final evaluation

FID drops rapidly from 46.76 to ~ 14 in the first 88 epochs, then gradually improves to **11.67** at epoch 388, after which it plateaus. Training continued for 302 more epochs without improvement before early stopping triggered.

8.4 Training Dynamics

Table 15 shows key training metrics at milestone epochs.

Table 15: Training dynamics at milestone epochs.

Epoch	Raw Loss	$\cos(V, v_c)$	$\cos(\hat{v}, v_c)$	Rel. Err.	$\ \nabla\ $	LR
0	2,788,299	0.083	0.170	1.353	1.384	1.0×10^{-4}
50	5,316,270	0.240	0.388	1.759	0.953	$\sim 9.9 \times 10^{-5}$
100	6,275,983	0.274	0.412	1.778	0.910	$\sim 9.7 \times 10^{-5}$
388	4,724,751	0.295	0.429	1.645	1.180	8.5×10^{-5}
689	4,327,238	0.307	0.436	1.516	0.610	5.6×10^{-5}

Key observations:

1. **Raw loss is not a good proxy for generation quality.** The raw loss is dominated by the MF term ($\mathcal{L}_{MF} \approx 6.5 \times 10^6$ vs $\mathcal{L}_{FM} \approx 7 \times 10^5$) and does not monotonically decrease. FID (computed from EMA-weight samples) is the correct early-stopping metric.
2. **Cosine alignment improves steadily.** $\cos(V, v_c)$ increases from 0.083 to 0.307; the v-head alignment converges faster ($0.170 \rightarrow 0.436$), confirming it provides good tangents from early training.
3. **Gradient norm decreases dramatically.** From 1.384 to 0.610; clipping fraction drops $\sim 90\% \rightarrow \sim 5\%$, indicating stable dynamics by epoch ~ 300 .

8.5 Generated Sample Statistics

The model’s output $\hat{x}$ statistics evolve during training (Table 16).

The standard deviation stabilises around 0.75–0.78, approximately 75% of the true latent std (~ 0.97 – 1.02). This **variance under-estimation** is the primary quality bottleneck: generated samples have slightly less contrast than real ones.

Table 16: Statistics of generated $\hat{x}$ at different training stages.

Epoch	$\mathbb{E}[\hat{x}]$	$\text{std}[\hat{x}]$	$\min[\hat{x}]$	$\max[\hat{x}]$
0	+0.004	0.243	−2.06	3.41
100	−0.001	0.737	−5.88	8.56
388	−0.001	0.753	−5.84	8.69
689	−0.001	0.781	−6.22	9.38

8.6 NFE Quality Analysis

Visual inspection of decoded samples reveals a clear quality hierarchy (Table 17).

Table 17: Visual quality at different NFE levels.

NFE	Quality	Key Features
1	Recognisable, blurry	Global structure correct, low contrast
2	Substantially sharper	Cortical sulci visible
5	Sharp, plausible	Clear grey-white contrast
10	Marginally crisper	Diminishing returns beyond 5

NFE-consistency metrics (shared noise, converged model): cosine similarity NFE=1 vs NFE=2: ~ 0.96 ; vs NFE=5: ~ 0.91 ; vs NFE=10: ~ 0.90 .

The 1-NFE output captures global anatomy correctly but lacks high-frequency detail—a known MeanFlow limitation. NFE=2 provides substantial improvement at only $2\times$ cost.

8.7 Spectral Analysis

The radially-averaged power spectrum evolves from flat (noise-like at epoch ~ 100) to the characteristic $1/f$ brain spectrum by epoch 600. All 4 channels show proper low-frequency dominance with high-frequency rolloff.

8.8 SWD (Sliced Wasserstein Distance)

SWD improves rapidly in early training (best = 1.779 at epoch 9) but then **diverges from FID**—SWD increases monotonically after epoch ~ 50 (final: 2.555) while FID continues to decrease. SWD is **not a reliable proxy** for perceptual quality in this setting.

8.9 Key Ablation Insights

x-pred vs u-pred. x-prediction with exact JVP trained stably through 690 epochs (best FID at 388). This supports the pMF manifold hypothesis: $d_{\text{input}}/d_{\text{bottleneck}} \approx 4$, firmly in the x-prediction regime.

x-pred + FD-JVP instability. Exponential growth of JVP norms ($10,000\times$ within 50 epochs). Root cause:

$$u = \frac{z_t - \hat{x}}{t} \implies \left. \frac{du}{dt} \right|_{\text{FD}} \approx \frac{u(t+h) - u(t)}{h} \sim \frac{O(1/t)}{h} = O\left(\frac{1}{t^2 h}\right) \quad (26)$$

Conditioning mode. (t, h) selected per MeanFlow Table 1c: FID 61.06 for (t, h) vs 63.13 for h -only.

Training stability fixes (Phase 4c–4e):

- β_2 : 0.999 $\rightarrow$ 0.95 (prevents stale gradients).
- Warmup: 5000 $\rightarrow$ 0 (destabilises MF loss).
- $\varepsilon_{\text{norm}}$: 0.01 $\rightarrow$ 1.0 (prevents runaway amplification).
- p_{norm} : 0.5 $\rightarrow$ 1.0 (sub-linear normalisation $\rightarrow$ gradient explosion).
- data_proportion: stabilised at 0.5 (50/50 FM/MF split).

9 Evaluation Protocol

9.1 Image Quality Metrics

Table 18: Evaluation metrics suite.

Metric	Description	Features
2.5D FID	Slice-wise FID (ax/cor/sag)	RadImageNet ResNet-50
3D FID	Volumetric FID	Med3D ResNet-50
MMD	Max Mean Discrepancy	Gaussian kernel
Coverage	Real–synthetic NN fraction	Feature-space NN
Density	Synthetic near real manifold	Complementary to coverage
MS-SSIM	3-level Multi-Scale SSIM (3D)	MONAI SSIMMetric
PSNR	Peak Signal-to-Noise Ratio	Actual data range

9.2 Spectral Analysis

High-frequency energy ratio via 3D FFT:

$$\rho = \frac{\sum_{|\mathbf{k}| > k_0} |\mathcal{F}(x)(\mathbf{k})|^2}{\sum_{\mathbf{k}} |\mathcal{F}(x)(\mathbf{k})|^2} \quad (27)$$

Reported for real, VAE-reconstructed, and generated volumes.

9.3 Morphological Assessment

SynthSeg [13] segmentation on both real and synthetic volumes: regional volume correlation, distributional KL divergence, and SynthSeg Dice overlap (paired by nearest-neighbour in feature space).

9.4 NFE Consistency Analysis

Generated samples at $\text{NFE} \in \{1, 2, 5, 10\}$ using shared noise, compared via per-sample L_2 distance and FID at each NFE level.

10 Implementation Details

10.1 Software Stack

Python 3.11.14, PyTorch 2.10+cu128, MONAI 1.5.2, PyTorch Lightning 2.6.1, OmegaConf/Hydra, HDF5 (h5py), Python logging with Rich handler + TensorBoard.

10.2 Gated Phase System

The project is implemented in 9 gated phases (0–8). Phase $N+1$ cannot begin until Phase N ’s critical tests all pass (Table 19).

Table 19: Phase status.

Phase	Title	Status	Tests
0	VAE Validation	Complete	7/7
1	Latent Pre-computation	Complete	7/7
2	Toy Experiment (Torus)	Complete	6/6 + 2 info
3	MeanFlow Loss + 3D UNet	Complete	28/28
4	Training on Brain MRI	Complete	12+
5	Generation + Evaluation	Code complete	11/11 local
6	Ablation Runs	Planned	—
7	LoRA Fine-Tuning (FCD)	Planned	—
8	Paper Figures	Planned	—

10.3 Compute Infrastructure

- **Local:** RTX 4060 8 GB — development, testing, analysis.
- **Picasso:** 4 nodes $\times$ 8 $\times$ A100-40GB — training, encoding, evaluation. SLURM with `-constraint=dgx`.

11 Novel Contributions

Contribution 1: First MeanFlow for 3D Medical Image Synthesis. We achieve 1-NFE generation of 192^3 brain MRI volumes in the latent space of a frozen MAISI VAE—a **50–1000 $\times$ reduction in sampling cost** compared to existing methods (DDPM: 1000; flow matching: 10–50; rectified flow: 5–50 steps). Best 2.5D FID: **11.67**, trained in $\sim$ 51 hours on 6 $\times$ A100 GPUs.

Contribution 2: Per-Channel L_p Loss in Latent MeanFlow. We extend the SLIM-Diff per-channel L_p loss [6] from pixel-space DDPM to latent-space MeanFlow, providing a framework for ablation over $p \in \{1.0, \dots, 3.0\}$.

Contribution 3: x-Prediction vs u-Prediction in Latent 3D UNets. First investigation of this dichotomy (established by pMF for 2D ViTs) in latent space with 3D UNets. We document the critical incompatibility: **x-prediction + FD-JVP is numerically unstable** due to the $1/t$ singularity.

Contribution 4: iMF Dual-Head in Medical Context. First application of the iMF dual-head architecture outside natural images. The v-head adds $\sim$ 228 K parameters ($<0.13\%$) and is disabled at inference.

Contribution 5 (Planned): LoRA Fine-Tuning for Joint Image-Mask Synthesis. Phase 7 will demonstrate LoRA fine-tuning for joint FLAIR MRI + FCD mask synthesis in a data-scarce setting ($\sim$ 50–100 cases).

12 Current Status and Next Steps

12.1 Current State (February 2026)

- **Phases 0–4:** Complete. Best model trained (x-pred + exact JVP, best FID 11.67 at epoch 388, early-stopped at epoch 690).
- **Phase 5:** Code complete, 11/11 local tests pass. Awaiting Picasso execution for latent generation at $\text{NFE} \in \{1, 2, 5, 10\}$, VAE decoding, and full quantitative evaluation.
- **Phases 6–8:** Planned.

12.2 Key Findings

1. **Early stopping was effective:** Best FID at epoch 388, used only 46% of planned 1,500 epochs. Total training: 51.4 hours.
2. **1-NFE quality gap is significant:** $\text{NFE}=1$ is blurry; $\text{NFE} \geq 2$ is substantially sharper.
3. **Variance under-estimation:** Generated std ~ 0.78 vs true ~ 0.97 – 1.02 . The 20–25% gap is the primary quality bottleneck.
4. **v-head converges fast:** $\cos(\hat{v}, v_c)$ reaches 0.39 by epoch 50 (85% of final 0.44).
5. **SWD anti-correlates with FID** beyond early training.
6. **Gradient clipping fraction:** $90\% \rightarrow 5\%$, confirming stable dynamics by epoch ~ 300 .

12.3 Open Questions

- Does the L_p exponent effect from pixel space transfer to latent space?
- How much does VAE smoothing degrade MeanFlow volumes vs multi-step methods?
- Can LoRA fine-tuning on ~ 50 – 100 FCD cases produce anatomically plausible lesion synthesis?
- Can the variance under-estimation ($\sim 75\%$ of true std) be mitigated by post-hoc rescaling or loss modifications?

- [1] Z. Geng, M. Deng, X. Bai, J. Z. Kolter, and K. He, “Mean Flows for One-step Generative Modeling,” *NeurIPS*, 2025 (Oral). arXiv:2505.13447.
- [2] Z. Geng, Y. Lu, Z. Wu, E. Shechtman, J. Z. Kolter, and K. He, “Improved Mean Flows: On the Challenges of Fastforward Generative Models,” arXiv:2512.02012, 2025.
- [3] Y. Lu, S. Lu, Q. Sun, H. Zhao *et al.*, “One-step Latent-free Image Generation with Pixel Mean Flows,” arXiv:2601.22158, 2026.
- [4] P. Guo, C. Zhao, D. Yang *et al.*, “MAISI: Medical AI for Synthetic Imaging,” *WACV*, 2025. arXiv:2409.11169.
- [5] C. Zhao, P. Guo, D. Yang *et al.*, “MAISI-v2: Accelerated 3D High-Resolution Medical Image Synthesis with Rectified Flow,” arXiv:2508.05772, 2025.
- [6] M. Pascual-González *et al.*, “SLIM-Diff: Shared Latent Image-Mask Diffusion with L_p Loss for Data-Scarce Epilepsy FLAIR MRI,” arXiv:2602.03372, 2026.
- [7] M. Yazdani *et al.*, “Flow Matching for Medical Image Synthesis,” *MICCAI*, 2025. arXiv:2503.00266.
- [8] Z. Dorjsembe *et al.*, “Semantic 3D Brain MRI Synthesis with Channel-Wise Conditioning,” *IEEE JBHI*, 2024.

- [9] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow Matching for Generative Modeling,” *ICLR*, 2023.
- [10] X. Liu, C. Gong, and Q. Liu, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow,” *ICLR*, 2023.
- [11] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-Resolution Image Synthesis with Latent Diffusion Models,” *CVPR*, 2022.
- [12] E. Hu, Y. Shen, P. Wallis *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models,” *ICLR*, 2022.
- [13] B. Billot *et al.*, “SynthSeg: Segmentation of Brain MRI Scans of Any Contrast and Resolution,” *Medical Image Analysis*, 2023.
- [14] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” *NeurIPS*, 2020.
- [15] R. Puglisi *et al.*, “Brain Latent Progression (BrLP),” *Medical Image Analysis*, 2025.